CSE422: ARTIFICIAL INTELLIGENCE

Section-11, Group 2

Project Report
on

Software Quality Analysis Using Machine Learning Models

Tasmiah Ahmed – 23201517
Anindita Mahjabin Sristy – 23201073

BRAC University, Fall 2025

INTRODUCTION

Software quality plays an important role in ensuring that modern software applications are reliable, maintainable, and successful. As software applications continue to grow in both size and complexity, the time and cost required to manually evaluate code quality and find possible defects increases significantly. Therefore, there is an increasing need for automated, data-driven approaches to evaluate software quality.

In this project, we used multiple machine learning techniques to examine various software development metrics to determine the quality of the software through the analysis of the dataset. The aim is to create **an automated software quality evaluation** by developing machine learning classifiers, which will allow early identification of low quality code, which helps the developers to make better decisions and improve the reliability and maintainability of the entire software system.

DATASET DESCRIPTION

The dataset contains **9 features** naming *Lines_of_Code, Cyclomatic_Complexity, Number_of_Functions, Code_Churn, Comment_Density, Number_of_Bugs, Has_Unit_Test, and Code_Owner Experience* and ***Quality label*** which is our target variable. The number of datapoints is 1600. There are 7 numerical features and 2 categorical features (Has_Unit_Tests, Quality Label). Quality label is a multi class feature (High, Medium, Low) and Has_Unit_Tests is a binary class (Yes, No).

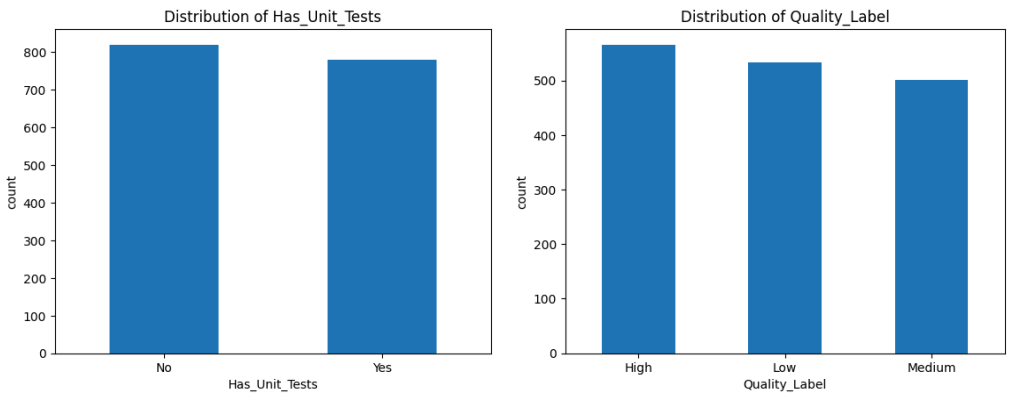
DATA ANALYSIS

At first, we performed Exploratory Data Analysis (EDA) on our dataset to explain the following important relationships from the data.

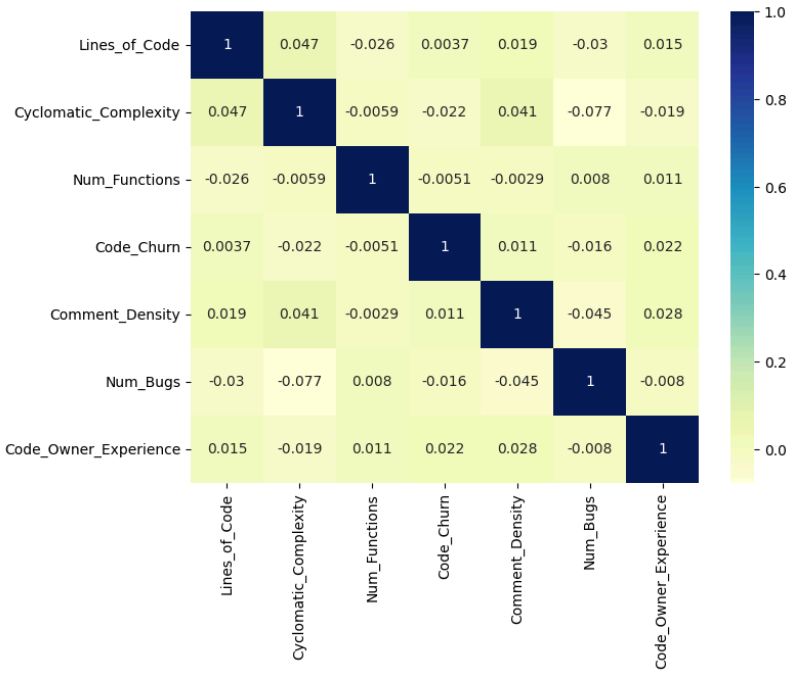
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1600 entries, 0 to 1599
Data columns (total 9 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Lines_of_Code          1520 non-null   float64
 1   Cyclomatic_Complexity  1600 non-null   int64
 2   Num_Functions          1600 non-null   int64
 3   Code_Churn             1520 non-null   float64
 4   Comment_Density        1520 non-null   float64
 5   Num_Bugs               1600 non-null   int64
 6   Has_Unit_Tests         1600 non-null   object
 7   Code_Owner_Experience  1600 non-null   int64
 8   Quality_Label          1600 non-null   object
dtypes: float64(3), int64(4), object(2)
memory usage: 112.6+ KB
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1600 entries, 0 to 1599
Data columns (total 9 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   Lines_of_Code          1600 non-null   float64
 1   Cyclomatic_Complexity  1600 non-null   int64
 2   Num_Functions          1600 non-null   int64
 3   Code_Churn             1600 non-null   float64
 4   Comment_Density        1600 non-null   float64
 5   Num_Bugs               1600 non-null   int64
 6   Has_Unit_Tests         1600 non-null   object
 7   Code_Owner_Experience  1600 non-null   int64
 8   Quality_Label          1600 non-null   object
dtypes: float64(3), int64(4), object(2)
memory usage: 112.6+ KB
```

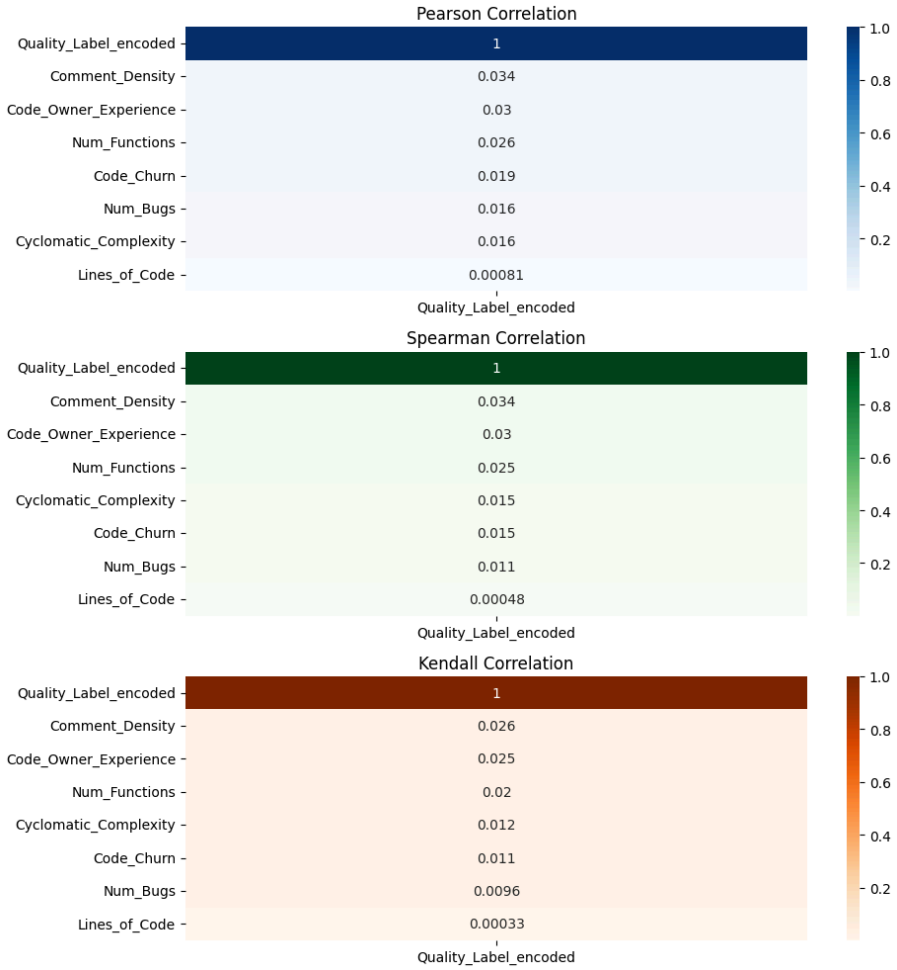
Dataset info before and after the processing of data



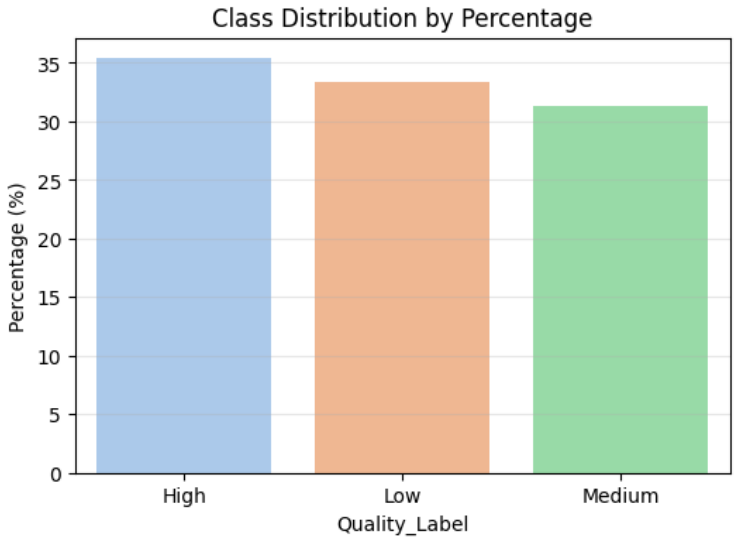
Barplot of unique value counts of categorical features



Correlation Heatmap Plot

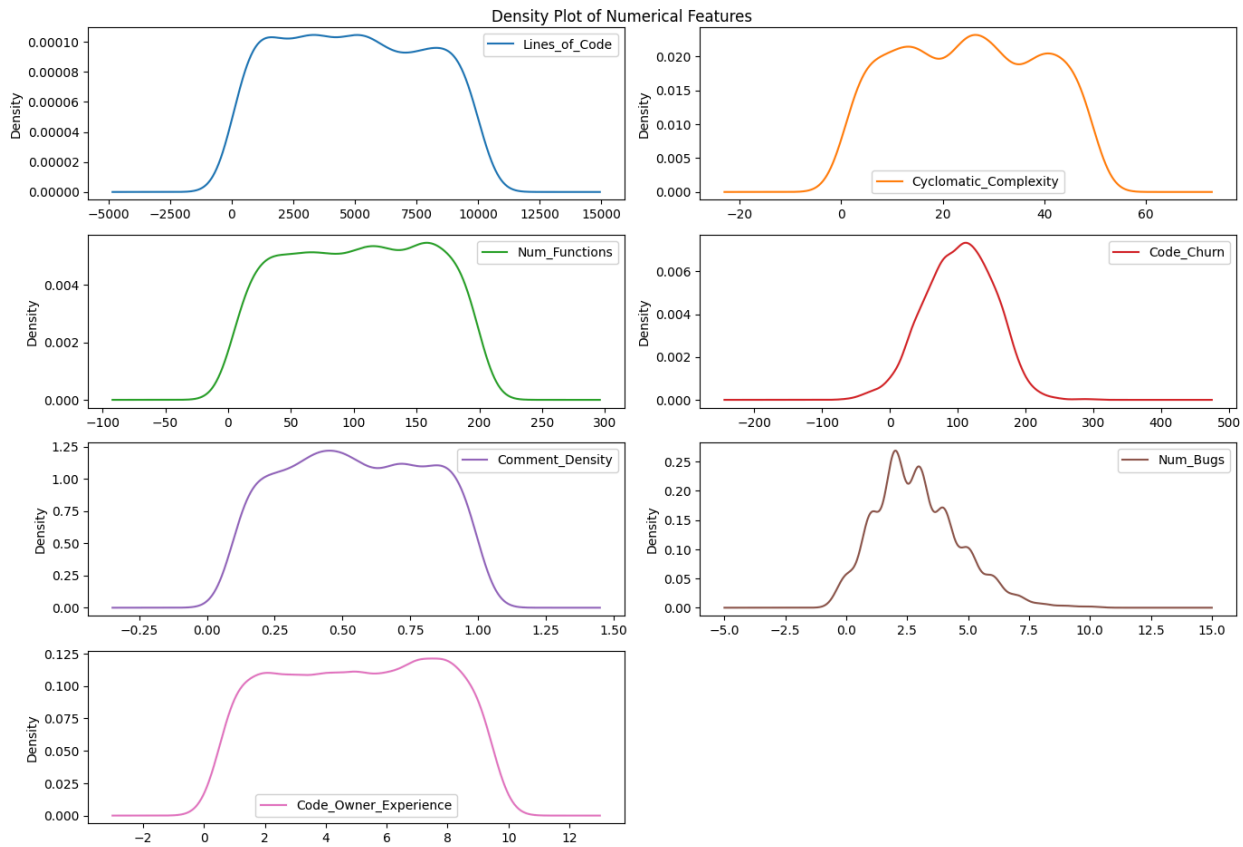


Correlation plot between numerical features and target



	Outcome	Count	Percentage
0	High	566	35.3750
1	Low	533	33.3125
2	Medium	501	31.3125

Barplot of Quality_Label by Percentage



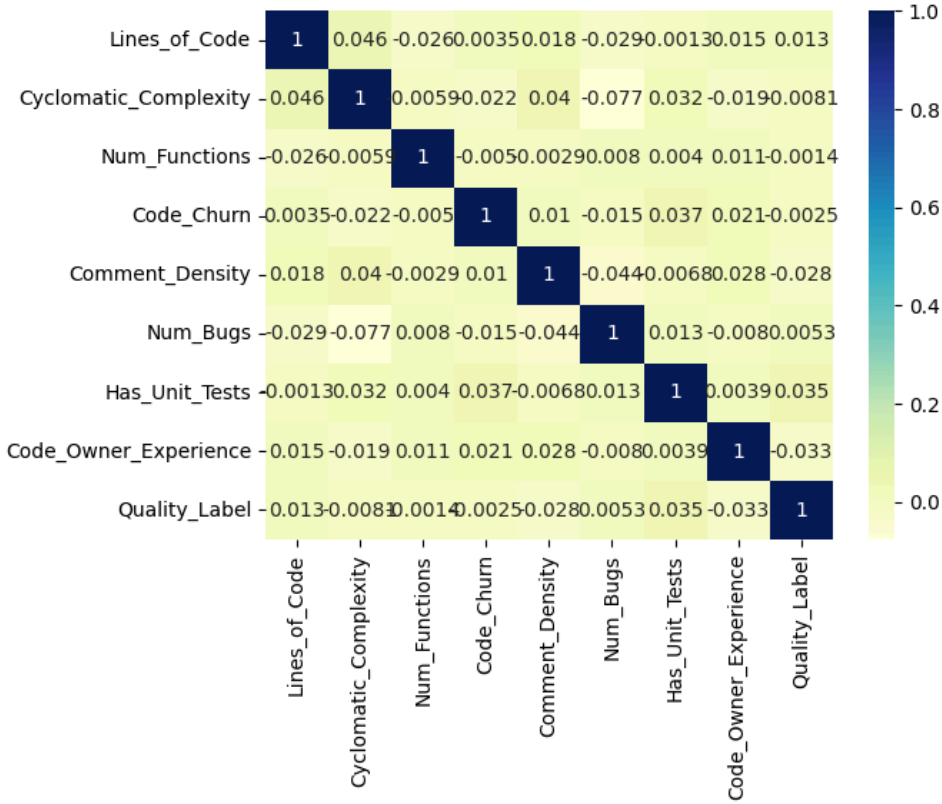
Density Plot of Numerical Features

We performed various exploratory data analysis techniques such as correlation, visualization on the given software quality analysis dataset. From the barplot of Quality_Label and Density Plot we can see the dataset is balanced and normally distributed. Analysing the insights acquired through EDA, we trained and tested our dataset through various Machine Learning Models.

DATASET PRE PROCESSING

Before starting the pre processing step, we splitted our data (1600 data points) into **Training Set** (1120 data points) and **Test Set** (480 data points). Next, in the pre processing part, firstly, we handled the null values. In our dataset, there were null values in– *Lines_of_Codes, Code_Churn and Comment_Density*. We handled the null values by **dropping rows** containing nulls. We also performed **Feature Engineering** to encode the categorical values by mapping. In **Feature Selection,** we analyzed the correlation heatmap, but as there were no features with high correlation, no column was dropped.

In **Feature Scaling,** we scaled our data by `MinMaxScalar` to ensure no feature dominates due to its range.



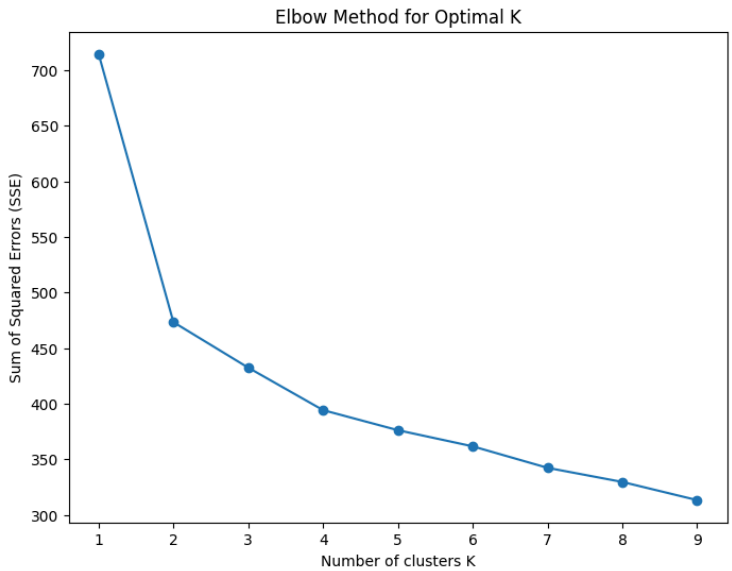
Correlation Heatmap

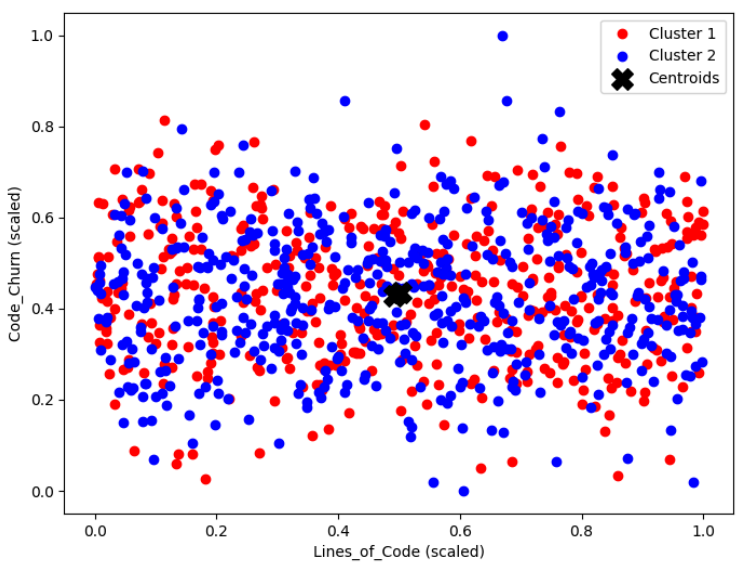
MODEL TRAINING

For model training, we applied 3 models: K- Nearest Neighbours(KNN), Neural Network (NN) and Gaussian Naive Bayes on the dataset as this is a supervised learning **classification problem** and the dataset is comparatively smaller. Accuracy of each model are KNN- 0.32, Naive Bayes- 0.32, MLP: 0.34. Then considering it as an unsupervised learning problem we applied K means clustering.

K means Clustering

We used the **Elbow Method** to find the number of clusters **K** by plotting the Sum of Squared Errors (SSE) for different K values. After selecting the value of K we applied **K-Means clustering** to assign each data point to a cluster. Finally, we visualize the clusters using two features and mark the centroids to see the grouping clearly. Here we cannot calculate accuracy like in supervised models so the clustering is to find groups in the data.

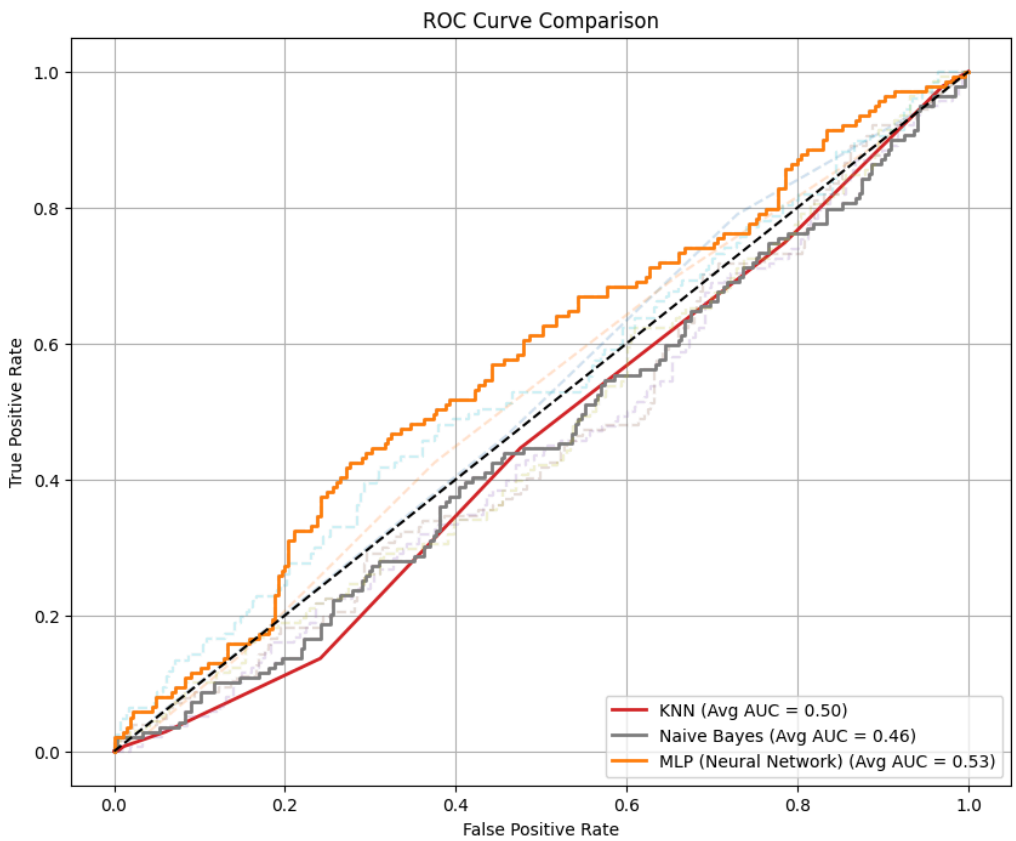




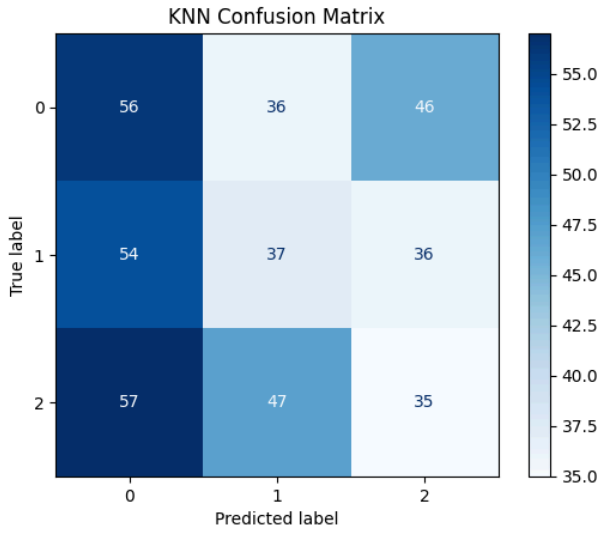
MODEL EVALUATION

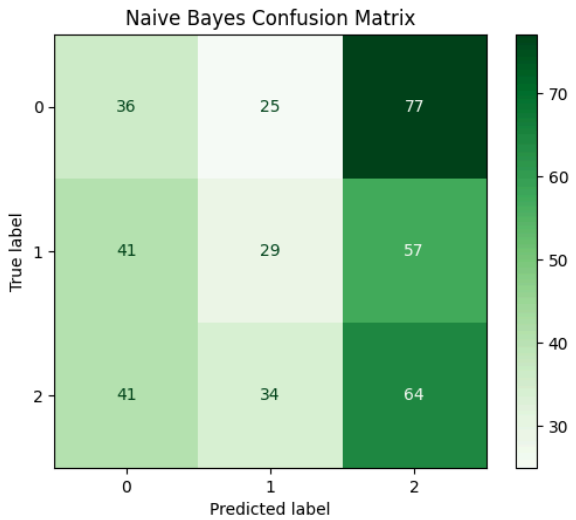
	Model	Accuracy	Precision	Recall	F1-Score
0	KNN	0.316832	0.314394	0.316832	0.313693
1	Naive Bayes	0.319307	0.319018	0.319307	0.311555
2	Neural Network	0.331683	0.326442	0.331683	0.313165

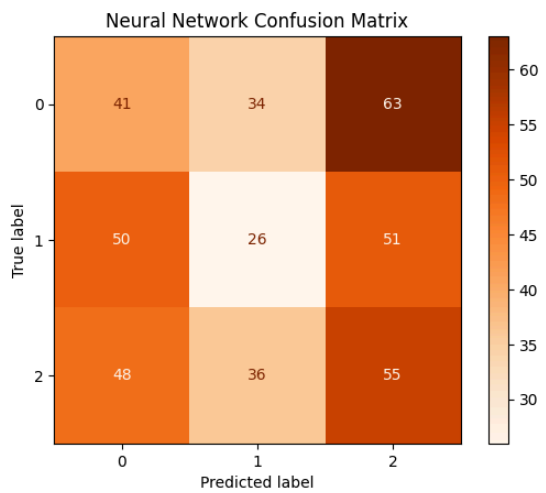
Model Performance Summary

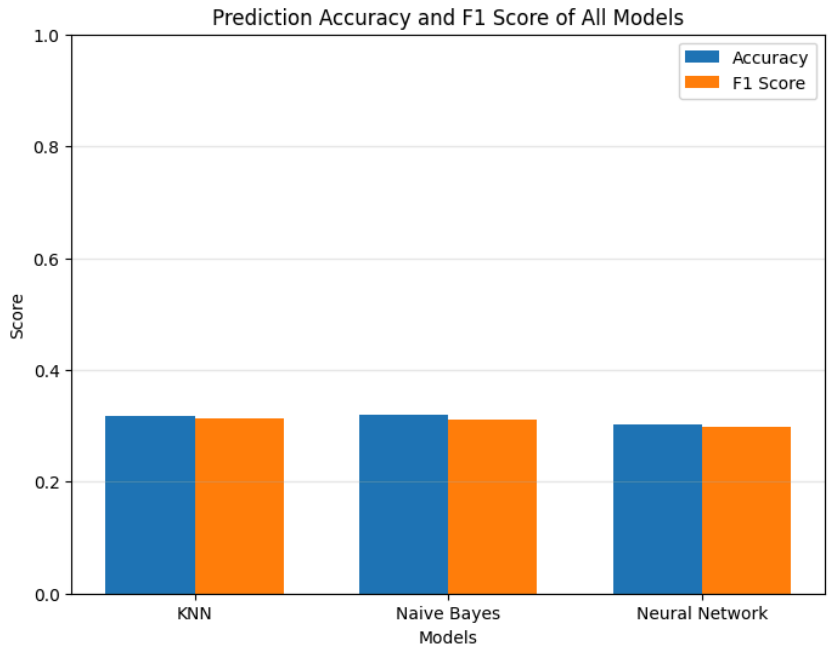


ROC Curve and AUC Score


KNN Confusion Matrix
True label
Predicted label
0
1
2
56
36
46
54
37
36
57
47
35
55.0
52.5
50.0
47.5
45.0
42.5
40.0
37.5
35.0


Naive Bayes Confusion Matrix
True label
Predicted label
0
1
2
36
25
77
41
29
57
41
34
64
70
60
50
40
30


Neural Network Confusion Matrix
True label
Predicted label
0
1
2
41
34
63
50
26
51
48
36
55
60
55
50
45
40
35
30



The performance of the three models K-Nearest Neighbors (KNN), Naive Bayes, and Neural Network (MLP) was evaluated using multiple metrics. The **bar chart** compares the accuracy and F1 scores of all models.These results indicate that the model with the highest accuracy is Neural Network, while the model with the highest F1 score is KNN.
The **confusion matrices** of each model reveal how well the models correctly predicted each class which highlights the ability of each model to distinguish between different classes.The **ROC curves** and **AUC scores** were calculated to evaluate the probabilistic performance of each model. The higher AUC value (closer to one) indicates how better the model performed and finally, in the Model Performance Summary we see how well each model is trained through various metrics.

CONCLUSION

Based on the evaluation metrics, (Model Performance Summary) the best performing model for this dataset is **Neural Network** due to its multiple layers and activation functions, such as ReLU and softmax. KNN performed well because of its distance-based approach, which works effectively in smaller datasets. Naive Bayes performed moderately because it assumes feature independence, which may not be true for all features in the dataset. Overall, the evaluation highlights the strengths and weaknesses of each model that helps us to analyse why certain models perform better and guide future decisions for model selection. `422 project Group 2.ipynb`